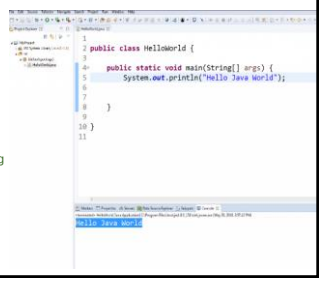
Lecture 1: Introduction to FP

1

What is Programming?

❖ **Program**

- A set of instructions that control the operation of a computer/machine to perform a specific task
- An executable software that runs on a computer
- A program is written using some programming language



2

What is Programming?

❖ **Programming Language**

- A programming language allows you to write programs, to code
- The primary role of the programmer is to construct a function to solve a given problem

What is a Programming Language?

- a tool for instructing machines
- a means of communicating between programmers
- a vehicle for expressing high-level designs
- a notation for algorithms
- a way of expressing relationships between concepts
- a tool for experimentation
- a means for controlling computerized devices

3

What is Programming?

❖ **Programming Language**

- Every non-trivial programming language provides
  - primitive expressions representing the simplest elements
  - ways to combine expressions
  - ways to abstract expressions, which introduces a name for an expression by which it can then be referred to

What is a Programming Language?

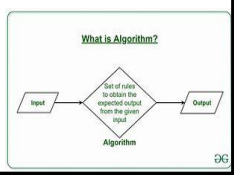
- a tool for instructing machines
- a means of communicating between programmers
- a vehicle for expressing high-level designs
- a notation for algorithms
- a way of expressing relationships between concepts
- a tool for experimentation
- a means for controlling computerized devices

4

What is Programming?

❖ **An algorithm**

- A routine which accepts a certain set of inputs, for each of which it gives an output after a finite number of steps
- Implemented by programs, that may be in the same or different programming languages
- Should
  - Terminate
  - Do what it is intended to do
- Has
  - **Preconditions** – assertions that must be valid before algorithm's execution
  - **Postconditions** – assertions that must be valid after the algorithm's execution



5

What is Programming?

❖ **Pseudocode**

- It is one of the methods which can be **used to represent an algorithm for a program**
- As compared to a program, pseudocode does not have a specific syntax therefore cannot be executed on a computer
- There are several formats which are used to write pseudo-codes
- Most of the formats take down the structures from languages such as C, Lisp, FORTRAN, etc

```
FUNCTION linearSearch(list, searchTerm):
    FOR index FROM 0 -> length(list):
      IF list[index] == searchTerm THEN
          RETURN index
      ENDIF
      ENDLOOP
          RETURN -1
END FUNCTION
```

6

Introduction to Functional Programming

What is Programming?

❖ **Pseudocode**

- Algorithms are presented using pseudocode since they can be read and understood by programmers who are familiar with different programming languages
- Pseudocode allows you to include several control structures such as While, If-then-else, Repeat-until, for and case, which is present in many high-level languages

➢ **Pseudocode is not an actual programming language**

```
FUNCTION linearSearch(list, searchTerm):
    FOR index FROM 0 -> length(list):
      IF list[index] == searchTerm THEN
          RETURN index
      ENDIF
    ENDLOOP
          RETURN -1
END FUNCTION
```

7

Introduction to Functional Programming

Programming Paradigms

- Programming languages can be categorized into **different paradigms**:
  - Object oriented
  - Imperative
  - Functional
  - Logic
- Each programming paradigm focuses on different aspects of programming, showing strength in some application areas but weakness in others

8

Introduction to Functional Programming

Object Oriented Languages

- OO exploits the mechanism of extending object classes to express the relationship between different **objects**
- Objects may contain data, in the form of fields, often known as **attributes**; and code, in the form of procedures, often known as **methods**
- Examples of OO languages are Java, PHP and C or C++
- An OO language must provide :
  - Abstraction
  - Inheritance
  - Encapsulation
  - Polymorphism

9

Introduction to Functional Programming

Object Oriented Languages

❖ **Abstraction**

- It refers to, providing only essential information to the outside world and hiding their background details
- That is, the ability to represent data at a very conceptual level without any details
- The idea is to deal with objects since their important features and ignore all other details
- Abstraction facilitates the easy conceptualization of real world objects, by eliminating the unnecessary details of the object

10

Introduction to Functional Programming

Object Oriented Languages

❖ **Encapsulation**

- This is a process of binding data members (variables, properties) and member functions (methods) into a single unit
- It is also a way of restricting access to certain properties or component
- The best example for encapsulation is a **class**
- The idea is to hide unnecessary details in classes and deliver a simple and clear interface for working
- Encapsulation provides a layer of security around manipulated data, protecting it from external interference and misuse

11

Introduction to Functional Programming

Object Oriented Languages

❖ **Inheritance**

- The ability to create a new class from an existing class is called inheritance
- Using inheritance, we can create a Child class(sub class) from a Parent class (super class) such that it inherits the properties and methods of the parent class and can have its own additional properties and methods
- That is, the sub class will have all the features of parent class and some of its own
- Class hierarchies develop code readability and support to the reuse of functionality

12

Introduction to Functional Programming

Object Oriented Languages

✧**Polymorphism**

- The word polymorphism means having many forms
- Typically, it occurs when there is a hierarchy of classes and they are related by inheritance
- Polymorphism allows an object to be processed differently by data types and/or data classes
- More precisely, it is the ability for different objects to respond to the same message in different ways
- It allows a single name or operator to be associated with different operations, depending on the type of data it has passed, and gives the ability to redefine a method within a derived class
- For example an operator can be overloaded so as to add two integer numbers and two floats
- Or, functions with same name with different functionality; saves time in investing many function names

13

Introduction to Functional Programming

Logic Programming Languages

- These are rule based languages
- The basis of logic programming is expressing facts and rules about some problem domain
- The rules are written as logical clauses with a head and a body; for instance, "H is true if B1, B2, and B3 are true."
- Facts are expressed similar to rules, but without a body; for instance, "H is true."
- Some languages are purely **declarative**, they allow for statements about what the program should accomplish, with no explicit step-by-step instructions on how to do so
- Others are a combination of **declarative and imperative**, they may also include procedural statements, such as "To solve H, solve B1, B2, and B3"

14

Introduction to Functional Programming

Imperative Programming Languages

- Also called **procedural**
- In imperative programming, the basic component is commands
- Typically, a program consists of a sequence of commands which yield the desired result when executed
- Programs in traditional languages, such as FORTRAN, Algol, C and Modula-3, rely heavily on modifying the values of a collection of variables, called the **state**
- The state is typically modified by assignment commands
  - ➢ often written in the form v = E or v := E where v is a variable and E some expression
- These commands can be executed in a sequential manner by writing them one after the other in the program, often separated by a semicolon

15

Introduction to Functional Programming

Imperative Programming Languages

- The **if** and **while** statements can be used to execute the commands conditionally, and repeatedly, depending on other properties of the current state
- The program amounts to a set of instructions on how to perform these state changes
  - a program consists of a sequence of commands which yield the desired result when executed
  - description of "**how to compute**" — how to sort an array, how to add an element to a linked list, how to traverse a tree, and so on
- These are the best-known imperative programming languages:
  - Fortran
  - ALGOL
  - C++
  - COBOL
  - Java
  - C
  - Assembler
  - Python
  - Pascal
  - C#
  - BASIC

16

Introduction to Functional Programming

Other Programming Languages

✧**Visual languages**

- A sub-category of imperative languages
- Provides simple ways to generate graphical user interfaces for programs
- Allows code segments to be dragged and dropped
- Examples: .NET

19

Introduction to Functional Programming

Other Programming Languages

✧**Scripting Languages**

- These are widely used in web applications
- It is where the script is written and where instructions for a run-time environment are written
- Scripts contain a series of commands that a software, application or scripting engine interprets one at a time within a runtime environment
- During runtime, these instructions are converted into readable machine code
- Programmers often use scripting languages to automate task execution within a runtime environment.
- Examples: Perl, JavaScript, Ruby

20

Introduction to Functional Programming

Other Programming Languages

❖ **Markup Languages**

- Not really programming languages
- A markup language uses tags to define elements within a document
- It is human-readable, meaning markup files contain standard words, rather than typical programming syntax
- Example: HTML for creating web pages , XML for storing structured date

21

Introduction to Functional Programming

Functional Programming Languages

- As its name suggests, FP emphasizes on the role of functions as the basic component of programs
- The basic concepts of functional programming originated from 2 lambda calculus
- Combined with proper support for modular development, functional programming proves to be an excellent choice for developing large-scale software
- Functional programming is often compared with imperative programming to highlight its characteristic features

22

Introduction to Functional Programming

Functional Programming Languages

- Functional programming naturally encourages programmers to concentrate on "**what to compute**" because every program fragment must have a value associated with it
- Functional languages emphasize the evaluation of expressions, rather than execution of command
- Using functions to combine the basic values forms the expressions in these languages
- All computation in a functional program is accomplished by applying functions to arguments
- A functional program can be considered as a mapping of inputs to outputs
- A mathematical function also maps domain elements (inputs) to range elements (outputs)
- For example, define a function f f : Domain --> Range, where Domain = {a, b, c}and Range = { p, q, r}. Let f(a) = r; f(b) = p; f(c) = q

24

Introduction to Functional Programming

Functional Programming Languages

- A functional program consists of an expression E (representing both the algorithm and the input)
- This expression E is subject to some **reduction rules**
- Reduction consists of replacing some part P of E by another expression P' according to the given reduction rules
- This process of reduction will be repeated until the resulting expression has no more parts that can be reduced
- The expression E" thus obtained is called the **normal form of E** and constitutes the output of the functional program

25

Introduction to Functional Programming

Functional Programming Languages

- Compared to imperative languages, there is no iteration (loops) in FP
  - There are **no "for" or "while" loop in functional languages**
  - Repetitive processes are specified with recursive function calls
  - Recursive functions repeatedly call themselves, until it reaches the base case
  - example of the recursive function:

```
fib(n)
   if (n <= 1)
      return 1;
   else
        return fib(n - 1) + fib(n - 2);
```

26

Introduction to Functional Programming

Functional Programming Languages

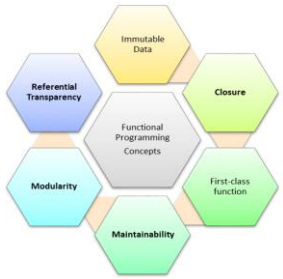
❖ **Examples of FP languages are:**

- LISP – the first FP language invented for list processing, mainly used in AI applications
- Scheme
- MetaLanguage (ML)
- Miranda
- Haskell
- SQL

27

Introduction to Functional Programming

Functional Programming Principles



28

Introduction to Functional Programming

Functional Programming Languages

- Functional Programming is **declarative**
- With declarative programming, code describes the task and abstracts away the logic
- On the contrary, imperative programming tells exactly how to do something in what steps

- FP is **result-oriented**
- That is, it is more concerned with what a task is and how it will end up, rather than with how to do it

29

Principles of FP

Modularity

- FP provides **modularity**
- Modular design increases productivity
- Small modules can be coded quickly and have a greater chance of re-use which surely leads to faster development of programs
- Apart from it, the modules can be tested separately which helps you to reduce the time spent on unit testing and debugging

30

Principles of FP

Maintainability

- FP offers **Maintainability**
- Maintainability is a simple term which means FP programming is easier to maintain as you don't need to worry about accidentally changing anything outside the given function

31

Principles of FP

Side Effects

- **Side Effects**
- Side effects are any state changes which occur outside of a called function
- The biggest goal of any FP language is to minimize side effects, by separating them from the rest of the software code
- In FP programming it is vital to take away side effects from the rest of your programming logic
- The principle of avoiding side effects brings the principle of **immutability i.e.** inputs can't be modified

32

Principles of FP

Variable Immutability

- **Variables are immutable**
- It is easy to create data structures instead of modifying ones which is already exist
- So, in FP, a variable can't be modified after it's been initialized
- Rather, than modify variables, new variables are created
- In FP, existing variables can't be modified, and this really helps to maintain state throughout the runtime of a program
- Once a variable is created and its value set, we can have full confidence knowing that the **value of that variable will never change**

33

Principles of FP

Variable Immutability

- If various assignments are required for the computation, new functions are created and feed off each other in a chain due to both the principle of immutability and the principle of avoiding side effect
- This is an another principle that can be called as **decompose;** decomposing the problem into functions

34

Principles of FP

Referential Transparency

❖ **Referential transparency**

- Functional programs should perform operations just like as if it is for the first time
- So, you will know what may or may not have happened during the program's execution, and its side effects
- In FP, variables once defined do not change their value throughout the program
- Functional programs do not have assignment statements
- If we have to store some value, we define new variables instead
- This eliminates any chances of side effects because any variable can be replaced with its actual value at any point of execution
- State of any variable is constant at any instant

```
x = x + 1 // this changes the value assigned to the variable x
          // So the expression is not referentially transparent
```

35

Principles of FP

First-Class and Higher-Order Functions

❖ **Functions are First-class and can be Higher-order**

- First-class function is a definition, attributed to entities that have no restriction on their use
- This means that a function is capable of standing alone and treated independently
- Therefore, first-class functions can appear anywhere in the program
- First-class functions are **treated as first-class variable**
- The first-class variables can be
  - passed to functions as parameter
  - returned from functions
  - stored in data structures

36

Principles of FP

First-Class and Higher-Order Functions

❖ **Functions are First-class and can be Higher-order**

- In FP, functions can be treated as variables
- So, functions can be passed into other functions and, they can be returned from other functions
- This is also called as ***functions as data***
- **Higher order functions** are the functions that take other functions as arguments, and they can also return functions as results

37

Principles of FP

Pure Functions

❖ **Pure functions** are used in FP

- A pure function will always produce the same output for the same arguments regardless of anything else
- Also, they have no side-effects, they don't do/change something other than your task inside a function
- A pure function doesn't create any external effects other than output value
- Pure functions do not modify any arguments or local/global variables or input/output streams
- Later property is called **immutability**
- The pure function's only result is the value it returns
- They are **deterministic**
- There should be no hidden inputs and they work like calculators e.g. 2+2 = 4 always the answer in calculator
- Pure functions are just like mathematical functions in this way

41

Principles of FP

Pure Functions

- In a pure function, only the input determines the output, nothing else can affect the output
- No matter how many times this function is called, it will always give the same output as long as the input is same
- A pure function guarantees that the input value remains unchanged
  - ➢ Input value is never modified
- The task of a pure function can be nothing more than calculating the output
- It can't read a file, or print something on somewhere, only the computation of output
- Its only interaction with the external system it works with is through the defined API
- This is in contrast to OOP, where object methods are designed to interact with the object's state (object members), and procedural style code, where the outer state is often manipulated from within the function

42

Principles of FP

Pure Functions

- Programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O
- Pure functions also make it easier to write parallel/concurrent applications
- When the code is written in this style, a smart compiler can do many things – it can parallelize the instructions, wait to evaluate results when needing them, and memorize the results since the results never change as long as the input doesn't change
- example

```
sum(x, y)     // sum is function taking x and y as arguments
   return x + y // sum is returning sum of x and y without changing them
```

43

Principles of FP

Pure Functions

```
//imperative function
def squaredSum(val1:Int, val2:Int) :
Double = {
     var sum:Int = val1 + val2
     var squared:Int = sum * sum
     return squared
}
println(squaredSum(3,4)) //49
```

```
//functional
def squaredSum(val1:Int, val2:Int) :
Double = {
     return Math.pow((val1+val2),2)
}
println(squaredSum(3,4))
```

- In first function, the intermediate steps to the final output are stored within variables before returning
- Creating such variables causes side-effects
- While doing the function operations, two new variables are also, *sum* and *squared*
- They have to be remembered and stored somewhere in the memory
- This means that the function produced variables or events outside of the its body and affected other locations
- A unit of scope affected scopes other than itself, pure functions don't do this

45

Principles of FP

Pure Functions

❖**Impure functions**

- Impure functions exactly in the opposite of pure
- They have hidden inputs or output
- Impure functions cannot be used or tested in isolation as they have dependencies
- Example

```
int z;
function notPure(){
z = z+10;
}
```

46

Principles of FP

Other Concepts

❖**Function Composition**

- Function composition is combining two or more functions to make a new one
- In mathematics, function composition is an operation $\circ$ that takes two functions ***f*** and ***g***, and produces a function $h = g \circ f$ such that $h(x) = g(f(x))$
- In this operation, the function g is applied to the result of applying the function f to x
- That is, first apply f (), then apply g ()
- We must also respect the domain of the first function
- Some functions can be de-composed into two (or more) simpler functions

47

Haskell

49

Introduction to Functional Programming

Haskell

- More at `http://www.haskell.org/`
- Named after Haskell B. Curry, one of the pioneers of lambda calculus
- First defined in 1990
- We will be using the **Glasgow Haskell Compiler interactive** GHCi (2010) interface, which is
  - freely available
  - Efficient and compact
  - Has a flexible user interface
- Haskell scripts use the **.hs** extension
- Haskell is a general purpose, purely functional programming language

50